

Using Particle Swarm Optimisations For Hyperparameter Tuning on LSTM Neural Nets

Lucy Dechaine - 700021039
Department of Computer Science
University of Exeter, Exeter, UK

Abstract—Particle Swarm Optimisations are a useful search algorithm for several search tasks. This paper explores their utility in searching for optimal hyperparameters for LSTM neural nets.

Keywords - Particle Swarm Optimisation, Long Short-Term Memory Networks, LSTM, Recurrent Neural Networks, RNN, Random Search, Stochastic Hill Climbing

I. INTRODUCTION

This paper explores the utility of Particle Swarm Optimisations for the task of optimising a set of hyperparameters for Long Short-Term Memory Networks. First, it takes a look at some key literature surrounding these topics, before exploring how these two topics interact. It looks in detail at how a particle swarm optimisation operates, and looks into key features of a LSTM network, and performs some tests to test how effective PSOs can be for this task. Finally, it suggests a range of areas where further research could be done in the goal of making PSOs more suitable to tuning LSTM hyperparameters.

II. LITERATURE REVIEW

A. Particle Swarm Optimisation

First introduced in 1995 by J. Kennedy and R. Eberhart [1], a Particle Swarm Optimisation (PSO) is an optimisation method designed for continuous, non-linear problems. It is modelled after animal swarming behaviour, like birds flocking or fish schooling, as well as taking inspiration from other areas, such as evolutionary programming and genetic algorithms.

The fundamental principle of the behaviour that various flock optimisations are based on is one of local inter-individual adjustments, where each individual in the population behaves in a way dictated by the individuals around it. Just like in cellular automata, even a simple set of rules for interacting with neighbours can lead to complex behaviour patterns arising.

The set of rules used for the initial simulation explored in the 1995 paper quickly converged on an unchanging direction, unlike the bird flocks it was designed to model. In order to add an element of random behaviour to the simulation, a parameter *craziness* was added that controlled the rate at which particles would take on a random direction, which resulted in more dynamic and "lifelike" behaviour. Then, this parameter was removed and replaced with the Cornfield vector, which is effectively a fitness function where the fitness is the distance between the particle and a target point (the paper uses $x=100$,

$y=100$), as a sum of the differences along both of the x-axis and y-axis:

$$Eval = \sqrt{(x_{present} - x_{target})^2} + \sqrt{(y_{present} - y_{target})^2}$$

Each particle aims to minimise this function, so it acts as a target for the particles to aim towards, thus approximating food. Each particle is also given the ability to "remember" its best known position, which it uses to adjust its velocity. This was done by checking which direction the particle needed to go to achieve this position again, and weighting the velocity by a small random amount in the calculated direction (no changes were made if the particle was at its best know position). This is scaled by the parameter $p_increment$, and can be thought of as similar to autobiographical memory as each particle remembers its own experiences. Additionally, each particle also "knows" the best known position of the entire population, which was used to modify the velocity in a similar fashion. The amount this is scaled by is the parameter $g_increment$, which functions like public knowledge, as every particle in the population knows of it. The weightings of these parameters can be set separately, which allows for tuning the strength of flock and individual behaviour. When these parameters are set high, the population is quickly drawn into the goal, and when they are set low, the flock slowly converged on the goal, exploring more of the search space in the process. The paper then suggests that approximately equal weighting between these two parameters provides the best results, as when $p_increment$ is set higher than $g_increment$, individuals tend to wander through search space excessively, and when $g_increment$ is set higher than $p_increment$, the swarm tends to converge too quickly on a local minima without exploring much of the remaining search space.

With the algorithm shown to work in 2-dimensional search space, the algorithm was then modified to work in N-dimensional space, with the x and y vectors replaced with arrays of length N, each entry representing a velocity in its respective dimension. This generalised algorithm was then applied to a non-linear multidimensional problem: tuning the weights of a multilayer perceptron neural network. It begins with a network with 2 input nodes, 1 output node, and 3 nodes in the hidden layer, where the PSO was used to tune the weightings between each node, giving $(2*3) + (3*1) = 13$ parameters to tune. The PSO was run in 13-dimensional space, where each dimension correlated to one parameter, and

achieved the target criteria of $e < 0.05$ in an average of 30.7 iterations when using 20 particles.

B. Long Short-Term Memory Networks

Recurrent Neural Networks (RNNs) are a type of neural network where the neurons are interconnected in both directions, allowing for signals from nodes to be sent backwards as a form of feedback [2]. They are well suited to many different types of problems, but are primarily designed for problems where the patterns occur in a sequential or time-varying order, such as computer vision tasks or text analysis, as this feedback can be used as a form of "short-term memory" [3] by reacting to recent input events, where the typical model weights function closer to "long-term memory" due to changing much slower.

However, these RNNs are prone to an issue known as the vanishing gradient problem, where error signals that are sent back through the network tend to either vanish completely or rapidly grow, depending on the size of the weights, causing the model to take far longer to train. The paper by S. Hochreiter and J. Schmidhuber [3] proposes a new architecture designed to tackle this issue, Long Short-Term Memory networks (LSTMs). It implements an efficient, gradient-based algorithm that confines the error flow to a constant space, preventing the explosive or vanishing effects the vanishing gradient problem is characterised by.

The paper first looks into previous methods of addressing the vanishing gradient problem. Many of the gradient-descent variants looked at are affected by the vanishing gradient problem [4] [5] [6] [7] [8]. Time-delays update unit activations based on previous activations by the unit, and can be used for short time lags [9]. Longer time lags can be managed with time constants, which changes the activation threshold of units based on the time step, however the time constants need tuning, and [10]. Rings approach deals with conflicting error signals by adding a higher order unit to influence the weightings between units. While fast, larger time steps may require larger numbers if units to be added, and does not generalise to unknown lag times. Several other methods were looked into, such as simulated annealing, multi-grid random search, and time-weighted pseudo-Newton optimisation [11], however these methods also ran into similar issues as before. Kalman filters try to reduce the exponential growth of previous dynamics, but they are assumed to be ineffective against small time lags over long periods. Second order nets uses multiplicative units (MUs), which is similar to how LSTMs operate, however these models do not enforce constant error flow, and such is not suited for long time lag problems. It also fully interconnects the MUs in its model, where LSTMs only use their MUs to control error signal flow.

The paper then explores how the exponential errors propagate through the network. It looks at a single node in the model, and looks into how the total error back flow is calculated, using mean squared as the error function. If the absolute value is greater than 1 for all weights, the error will grow exponentially, leading to the model training with oscillating weights and unstable learning. If the absolute value is less

than 1, however, the error decreases exponentially, quickly becoming irrelevant to the models training, hence learning rates slow drastically. If the function used for error is instead the sigmoid function, the model is still incredibly sensitive to error values, and as the relevant derivative converges on 0 at a faster rate than the absolute weight can grow, the error tends to vanish quickly, even with larger initial weights.

Then the paper explores a naive approach to a constant error flow, involving enforcing constant error flow by designating an error connection connecting each node to itself, and setting the magnitude to be exactly 1, thus preventing exponential growth and decay at each node. Due to this, the fitness function is required to be linear, and each units activation threshold must be constant. As each unit is also connected to other units in the model, this may cause some conflicts with both the input and output weights. As incoming weights are used to both store some inputs and ignore others, the conflict causes learning to be more difficult, and requires a better method for separating the effect of these two types of weighting updates. Furthermore, the units output is also used in two different conflicting ways, as retrieving the units content to pass on to the next unit, as well as to prevent the input stored in the unit from propagating onwards to the next unit. These conflicts tend to become more noticeable with long time lags, but do affect small time lags too.

The LSTM architecture introduces memory cells and gate units, in which each unit has two components, the linear unit as typically found in other NNs, and the self connection (similar to as found when enforcing constant error backflow). The relationship between these components comes with a multiplicative input gate, which protects the value stored in the unit from irrelevant changes, and a multiplicative output gate, which prevents the value stored from propagating irrelevant changes itself. This unit is known as the memory cell. The gates in the memory cell operate similarly to a storage latch, where the value remains unchanged until a suitable signal is sent to change it, however this means there are more parameters for the model to tune during training. Output gates may be unnecessary to the model though, as preventing memory cells from propagating error changes can be managed with only input gates, but output gates may still be helpful as they assist with learning long time lag memories.

III. MAIN BODY

A. Problem Formulation

Long Short-Term Memory Neural Networks, similarly to many other types of neural networks, utilise a set of parameters to fine tune the behaviour of the network, called hyperparameters. In order for the network to achieve the best performance, both in training speed and trained accuracy, these hyperparameters will require tuning. As there are many hyperparameters that have varying effects that interact with each other, it is not a straightforward task to pick the optimum set of hyperparameters, and often requires multiple iterations to get desirable results. As these hyperparameters can be expressed as a list of variables, it is possible to represent

these as a search space, and use an optimisation algorithm to automate the process of hyperparameter tuning.

B. Search Algorithm : Particle Swarm Optimisation

The Particle Swarm Optimisation algorithm is modelled after a set of point-like particles that do not interact with their local neighbours. Each particle individually has a position in search space, a velocity, a memory of its best known position, and knowledge of the best position any particle has achieved. The particles update their velocity based on its best known position and the global best known position, according to their weightings.

The following pseudocode shows the Particle Swarm Optimisation algorithm. It initialises the population with random positions distributed uniformly across search space and random velocities within velocity space (where velocity space has twice the magnitude as search space in every direction, accounting for positive and negative velocity components). The best solution across the population according to the fitness function is then noted, and the algorithm begins iterating.

In each iteration, each particle's velocity is updated using the best known positions, scaled by their weightings as well as a random factor between 0 and 1, generated randomly for each direction component. This allows the velocities to explore more of the search space, while still tending to the optimal solution. The position is then updated based on the new velocity, and the new position is checked against the particle's previous best position, and then the global best if a new best is found. Iterations repeat until the termination criteria is met, which in this case is a pre-determined number of iterations.

Initialisation

```

for particle  $p$  in population do
  for direction in  $p$  do
     $p.position \leftarrow Random \sim U(d_{min}, d_{max})$ 
     $p.velocity \leftarrow Random$ 
     $\sim U(-\|d_{max} - d_{min}\|, (d_{max} - d_{min}))$ 
  end for
   $p.best \leftarrow p.position$ 
  if  $Fitness(p.best) > Fitness(gbest)$  then
     $gbest \leftarrow p.best$ 
  end if
end for

```

Searching

```

for Iteration  $i$  do
  for particle  $p$  in population do
    for direction  $d$  in  $p$  do
       $r_p, r_g \leftarrow Random\ U(0, 1)$ 
       $p_d \leftarrow w * p_d$ 
       $+ cc * r_p * (p.best_d - p.position_d)$ 
       $+ sc * r_g * (gbest_d - p.position_d)$ 
       $p.position = p.position + p.velocity$ 
    end for
    if  $Fitness(p.position) > Fitness(p.best)$  then
       $p.best \leftarrow p.position$ 
    if  $Fitness(p.best) > Fitness(gbest)$  then

```

$gbest \leftarrow p.best$

end if

end if

end for

end for

As discussed previously in section II-A, a PSO will perform best when the weightings between the particle's best and the global best positions are roughly equal, and when neither are too large or too small, leading to the PSO having parameters that will also require tuning. Furthermore, the number of particles and the number of iterations must also be tuned in order to get the optimal performance.

C. Adaptation of Algorithm to Problem

For this implementation of a LSTM, there are 5 parameters that have been encoded in a way the PSO can explore. These five parameters are the size of the hidden layers in the model, the number of layers in the model, the dropout rate, the learning rate, and the number of epochs. These parameters are represented in search space as floats between 0 and 1, which gives the PSO a uniform search space to explore efficiently.

Two of these parameters, namely the dropout rate and the learning rate, do not need any further modification as they are also required to be floats between 0 and 1, however the other three parameters will need more processing. They are required to be integers, which is achieved by multiplying the float value by scale factor and rounding it to the nearest integer before passing it into the LSTM model. This integer should be sufficiently large enough to cover a suitable space, for best exploration of epoch, node, and layer counts, however due to the time-intensive fitness function, as discussed later next, setting it too large may cause the exploration of the search space to take prohibitively long.

To explore the effects of changing these parameters, the PSO algorithm is applied over the 5 dimensional search space, and each candidate solution is evaluated using a fitness function, where the aim is to minimise the value obtained from that function. The fitness function used first applies the parameters from the candidate to a new LSTM model and uses them to train the model, using some artificially generated synthetic data designed for LSTM training. The result of the fitness function is then calculated as the mean squared error of the models predictions over data generated for the synthetic dataset that has not been used for training the model.

D. Experiments

In order to analyse the utility of a Particle Swarm Optimisation for tuning hyperparameters of a LSTM model, the algorithm was compared to both a random search algorithm, and a stochastic hill climber. A random search generates a random set of values each iteration, giving a comparison to a good baseline uninformed algorithm. The stochastic hill climber picks a random index in the candidate to perturb, taking the new change if it is an improvement over the previous solution. This is a good baseline informed algorithm for a comparison point. As neither random search nor stochastic hill

climber use a population of individuals, to keep the number of individuals tested across the algorithms consistent.

The dataset used consisted of synthetic data, created using a periodic function over a time frame (oscillating once per synthetic year), a linear trend, and a small random noise, as shown below 7. When creating the data, the number of oscillations is modifiable, and a value of 30 was chosen for this, as it is suitably large to demonstrate the time lag features of a LSTM, but not too large to be noise itself. This parameter may be possible to optimise, however this was not seen as a priority.

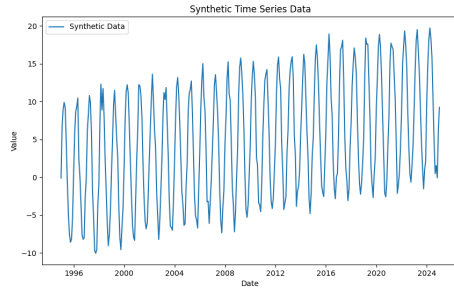


Fig. 1. Synthetic Dataset

The first set of tests used a small sample, with 3 candidates per generation, and 10 generations, for 30 individuals per experiment. In order to keep this test less time intensive, this test was performed with an integer scale factor of 19, ensuring a maximum of 20 epochs, 20 hidden layers and 20 nodes per layer (and a minimum of 1 of each). The parameters for the PSO were also set to an inertia weighting of 0.7, and both the social and cognitive weightings to 1.5. These values were chosen based on rough guidelines and intended to be a starting point to iterate upon. The results of each test are graphed below.

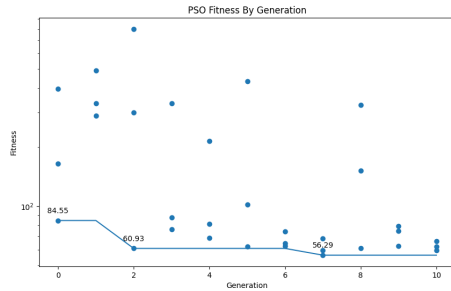


Fig. 2. Particle Swarm Optimisation Experiment 1

In this experiment, we see the Particle Swarm Optimisation algorithm producing weights that correspond to a hidden layer size of 12, 13 layers, 12 epochs, a dropout rate of 0.866, and a learn rate of 0.101, and achieved a mean squared error of 56.3. The Random Search algorithm found weights corresponding to a hidden layer size of 17, 9 hidden layers, 11 epochs, a dropout rate of 0.564, and a learn rate of 0.807, achieving

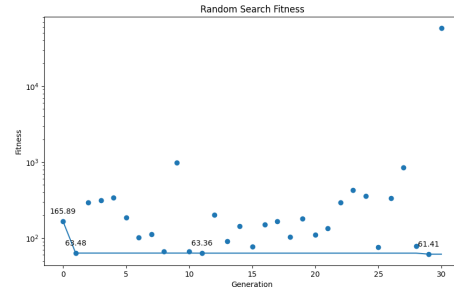


Fig. 3. Random Search Experiment 1

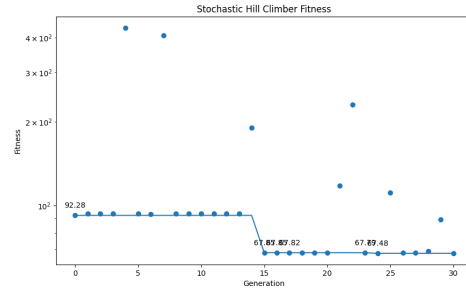


Fig. 4. Stochastic Hill Climber Experiment 1

a mean squared error of 61.4. The Stochastic Hill Climber produced weights corresponding to a hidden layer size of 6, 3 hidden layers, 3 epochs, a dropout rate of 0.404, and a learn rate of 0.136, while achieving a mean squared error of 67.5.

In this experiment, despite the sample size being relatively small, we can see the PSO beginning to converge on a solution, and making steady improvements. The stochastic hill climber, in comparison, makes only one improvement across the iterations, and the random search does not converge at all. The PSO also provides the best solution in this, although as all three solutions are in the same ballpark, which suggests further tests should be carried out to explore their limits.

The second test involved increasing the search space, with the integer scale used being 39 instead of 19, doubling the range of the integer parameters, as well as increasing the iterations to 15 and population size to 5. This leaves 75 individuals in each population, and the social and cognitive weightings for the PSO have been increased to 1.8 to encourage slightly faster converging. The results for these tests are found below.

The second test produced results of varying quality. the stochastic hill climber performed notably better than it did in the first test, achieving a mean squared error of just 15.9, with weights correlating to a hidden layer size of 21, 4 layers, 13 epochs, a dropout of 0.280, and a learn rate of 0.0057. It was able to use the larger search space effectively, and climbed to near the top of its local maxima effectively.

Random search achieved a mean squared of 47.6, using a hidden layer size of 13, 16 layers, 26 epochs, a dropout rate of 0.963, and a learn rate of 0.459, and on account of it being an

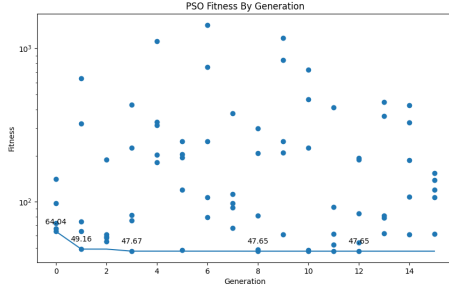


Fig. 5. Particle Swarm Optimisation Experiment 2

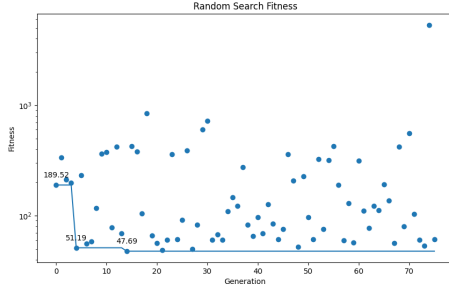


Fig. 6. Random Search Experiment 2

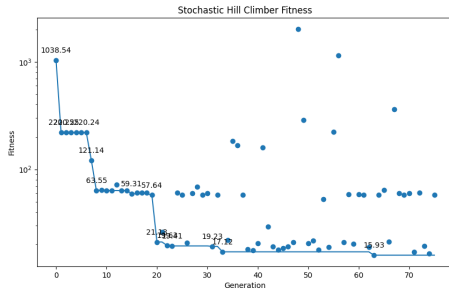


Fig. 7. Stochastic Hill Climber Experiment 2

uninformed search, it did not show any convergence, however it does show that the search space is roughly contained between approximately 50 and 10^3 .

The Particle Swarm Optimisation, however, performed significantly worse than expected. Instead of converging on a more optimal solution, it produced a result nearly identical to a random search, with a mean squared of 47.6 , less than 0.04 better. Additionally, the spread of data points tends to be slightly higher values than the random search. While this would suggest that the search algorithm has failed to converge, a closer look into the values the search algorithm has pushed suggests otherwise. The final results from this test was the maximum integer of 40 for all three of layer count, hidden layer size, and epochs, and this was the case even early on in the searching. During searching the other two values, for dropout rate and learn rate, fluctuated around 1 and 0

respectively, producing a dropout rate of 0.866 , and a learn rate of 0.154 at their best value. Because of the trend towards a larger model for a relatively small task, the models these parameters produce are heavily volatile and are not good models. Furthermore, the large size of these models caused the evaluation process to take significantly more time than either of the other search methods.

E. Future Work

While exploring this topic, one main area for future improvement has become increasingly clear. Due to the incredibly time intensive fitness function to evaluate the search space against, it would be useful for future work to develop more advanced methods for reducing the time taken for each evaluation. There are many different ways this could be achieved, but some methods may be more effective than others.

One method for this that could be explored is particle culling. As evaluating the fitness function is time intensive, while each model is training it may be possible to predict if the model is badly tuned, for example using the loss functions rapidly increasing in value as an indicator, and discarding the model as a bad data point. Doing this process early in the training of a model with bad parameters may save a lot of time, however doing it too early may add a risk of removing good models that start training slowly. This method may also become less effective with longer search algorithms, as it will be harder to remove bad solutions when converging on good solutions as it will be more likely that any solution being evaluated is suitably good.

Similarly, there could be detection in place for when a search algorithm has converged on a solution, as if the optimal solution has been found early on, it is not necessary for the same solution to be tested for the whole search algorithm if there are no better solutions that the algorithm can find.

This may also be achieved by implementing a time limit for training, where models must finish training within a set time limit or be evaluated before every epoch has been completed. This would be a more effective strategy for longer search algorithms, however may cause instability with the outputs it generates.

Another method for achieving this may be to introduce a parameter to account for the time taken in the fitness function. This could be a simple linear factor, either multiplying or summing the current fitness function with the time taken, or a more complex function involving both values. This would provide some incentive for the search algorithms to explore the trade-off between training time and prediction accuracy, as well as possibly reducing the chance of the model over-fitting.

Another option for future work would be evaluating the fitness against slightly different prediction data, as the static nature of using the same data will introduce unnecessary risk of over-fitting. This could be further expanded into training the models to deal with different number of periodic cycles, as this paper has not explored the models predictive power over different ranges.

IV. CONCLUSION

While a particle swarm optimisation may seem applicable to the problem of tuning hyperparameters of a LSTM neural network, this paper has explored some ways in which it requires more research into utilising it effectively. There are several ways this could be explored in the future, and a PSO could at least in theory be useful in some similar situations with these developments explored.

REFERENCES

- [1] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948 vol.4.
- [2] L. R. Medsker, L. Jain *et al.*, "Recurrent neural networks," *Design and Applications*, vol. 5, no. 64-67, p. 2, 2001.
- [3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, 11 1997.
- [4] J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/036402139090002E>
- [5] S. Fahlman and C. Lebiere, "The cascade-correlation learning architecture," *Advances in Neural Information Processing Systems*, vol. 2, 10 1997.
- [6] Pearlmutter, "Learning state space trajectories in recurrent neural networks," in *International 1989 Joint Conference on Neural Networks*. IEEE, 1989, pp. 365–372.
- [7] J. Schmidhuber, "A fixed size storage $O(n^3)$ time complexity learning algorithm for fully recurrent continually running networks," *Neural Computation*, vol. 4, no. 2, pp. 243–248, 1992.
- [8] R. J. Williams, "Complexity of exact gradient computation algorithms for recurrent neural networks," Technical Report Technical Report NU-CCS-89-27, Boston: Northeastern . . . , Tech. Rep., 1989.
- [9] K. J. Lang, A. H. Waibel, and G. E. Hinton, "A time-delay neural network architecture for isolated word recognition," *Neural networks*, vol. 3, no. 1, pp. 23–43, 1990.
- [10] M. C. Mozer, "Neural net architectures for temporal sequence processing," in *Santa Fe Institute Studies in the Sciences of Complexity- Proceedings Volume-*, vol. 15. Addison-Wesley Publishing Co, 1993, pp. 243–243.
- [11] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE transactions on neural networks*, vol. 5, no. 2, pp. 157–166, 1994.